

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Объектно-ориентированное программирование»

«К защите допустить»
Руководитель курсового проекта
ассистент кафедры информатики
_____. А.Г. Бокун
_____. _____. 2026

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
курсового проекта
на тему:
«КОРПОРАТИВНЫЙ МЕССЕНДЖЕР»

БГУИР КП 6-05-0612-02 014 ПЗ

Выполнил студент группы 453501
ЛАПУНОВ Иван Андреевич

(подпись студента)

Курсовой проект представлен на
проверку _____. _____. 2026

(подпись студента)

Минск 2026

СОДЕРЖАНИЕ

Введение.....	5
1 Постановка задач для реализации проекта.....	6
1.1 Анализ существующих аналогов.....	6
1.2 Анализ предметной области	8
2 Технологии программирования, используемые для решения поставленной задачи	10
2.1 Язык программирования и серверная платформа	10
2.2 Технологии взаимодействия клиента и сервера	11
2.3 База данных и работа с данными.....	13
2.4 Архитектура программного обеспечения.....	13
2.5 Клиентская часть приложения.....	15
2.6 Обеспечение безопасности	16
3 Разработка программного приложения	17
3.1 Общая архитектура системы.....	17
3.2 Архитектура серверной части.....	18
3.2.1 Структура проекта	18
3.2.2 Реализация паттернов проектирования	22
3.2.3 Реализация API (эндпоинты)	24
3.2.4 Модель базы данных.....	26
3.3 Реализация клиентской части	27
3.3.1 Общая структура фронтенда.....	27
3.3.2 Пользовательский интерфейс	28
4 Тестирование	33
Заключение	33
Список использованных источников	35

ВВЕДЕНИЕ

В условиях стремительного развития информационных технологий и цифровизации бизнеса особое значение приобретает организация эффективного взаимодействия между сотрудниками внутри компании. Современные организации всё чаще используют удалённые и гибридные форматы работы, что требует внедрения надежных и удобных средств коммуникации. Одним из ключевых инструментов в этом процессе являются мессенджеры, позволяющие оперативно обмениваться информацией, координировать действия и поддерживать рабочие процессы.

На сегодняшний день существует множество популярных решений для обмена сообщениями, однако большинство из них ориентированы на массового пользователя и не учитывают специфику корпоративной среды. Использование сторонних мессенджеров связано с рядом ограничений, включая зависимость от внешних сервисов, невозможность полного контроля над передаваемыми данными, а также риски, связанные с безопасностью и конфиденциальностью информации. В некоторых случаях использование таких решений может быть ограничено внутренними политиками организации или требованиями законодательства.

Кроме того, существующие решения часто обладают избыточным функционалом, который не только не используется в рабочем процессе, но и может снижать продуктивность сотрудников. В корпоративной среде приоритетом являются простота, надежность, безопасность и возможность адаптации системы под конкретные задачи организации.

В связи с этим возникает необходимость разработки специализированного корпоративного мессенджера, который обеспечит централизованное управление пользователями, контроль данных, а также будет обладать необходимым функционалом для эффективной коммуникации.

Целью данной курсовой работы является разработка корпоративного веб-мессенджера, предназначенного для внутреннего использования сотрудниками организации. Для достижения поставленной цели необходимо провести анализ существующих аналогов, исследовать предметную область, определить основные функциональные требования, выбрать подходящие технологии разработки и спроектировать архитектуру системы.

Пояснительная записка оформлена в соответствии с СТП 01-2024 [1].

1 ПОСТАНОВКА ЗАДАЧ ДЛЯ РЕАЛИЗАЦИИ ПРОЕКТА

1.1 Анализ существующих аналогов

В настоящее время рынок программных решений для обмена сообщениями представлен большим количеством приложений, которые используются как в повседневной жизни, так и в профессиональной деятельности. Однако не все из них подходят для использования в корпоративной среде, поскольку не обеспечивают необходимый уровень контроля, безопасности и адаптации под внутренние процессы организации.

Одним из наиболее известных корпоративных мессенджеров является Slack, который широко используется в IT-компаниях и стартапах. Данное приложение предлагает удобную систему каналов, позволяющую структурировать коммуникацию внутри команды, а также поддерживает интеграцию с большим количеством внешних сервисов. Несмотря на это, Slack имеет ряд недостатков, среди которых можно выделить зависимость от облачной инфраструктуры, ограниченные возможности настройки под конкретные нужды организации и платную модель использования при увеличении числа пользователей.

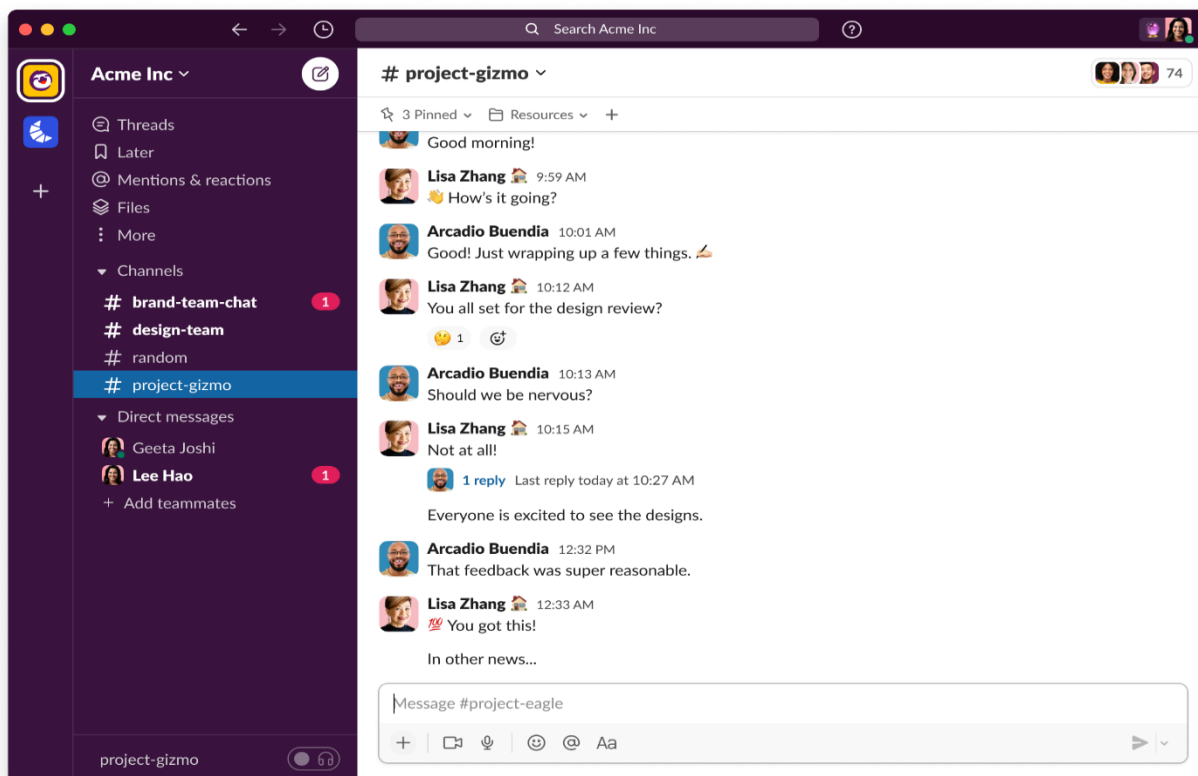


Рисунок 1.1 – Slack

Другим популярным решением является Microsoft Teams — комплексная платформа для совместной работы. Она объединяет мессенджер, видеоконференции и глубокую интеграцию с Office 365. Однако запутанная навигация по каналам и вкладкам, чрезмерное потребление оперативной памяти и перегруженность лишними инструментами делают сервис тяжеловесным. Для небольших команд или простых рабочих чатов Teams часто оказывается избыточным и медлительным решением.

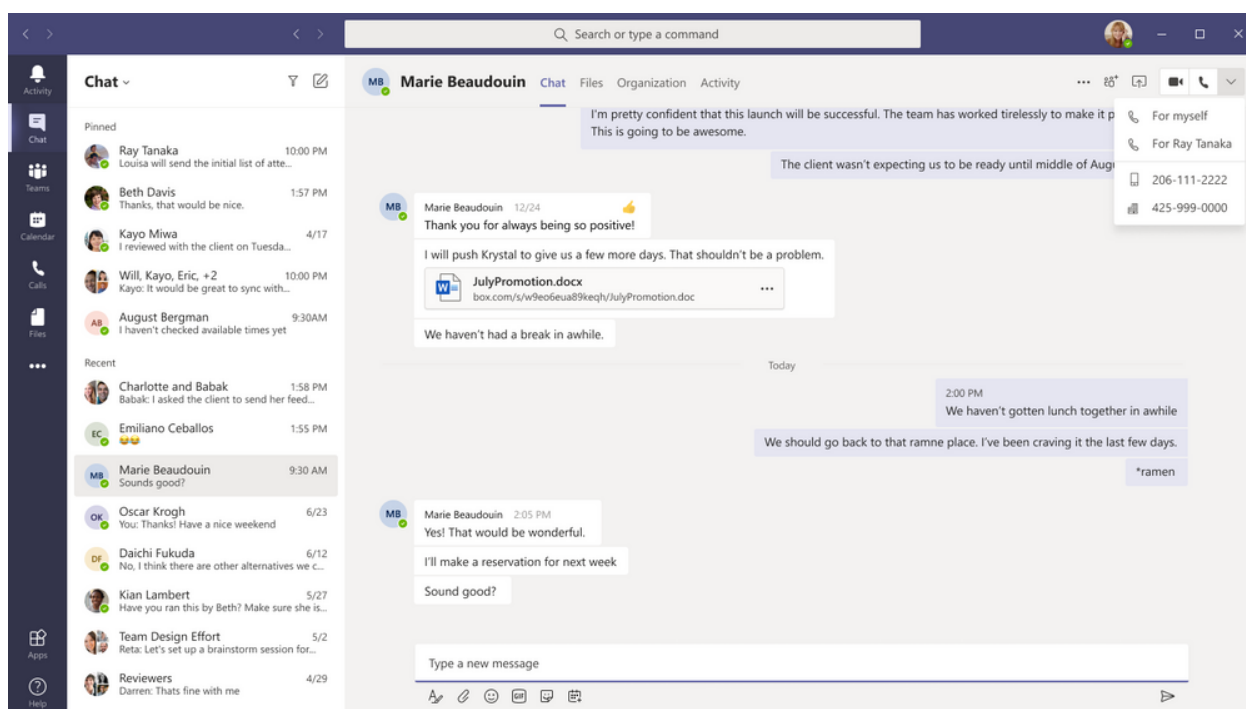


Рисунок 1.2 – Microsoft Teams

В качестве альтернативы часто выбирают универсальные мессенджеры, такие как Telegram. Он подкупает высокой скоростью работы, интуитивным интерфейсом и мощными групповыми чатами. Тем не менее Telegram не является полноценным корпоративным решением: в нем данные хранятся на сторонних серверах без возможности локального контроля. Кроме того, риски блокировок и замедления трафика (особенно актуальные в РФ) делают его нестабильным инструментом для критически важных бизнес-процессов.

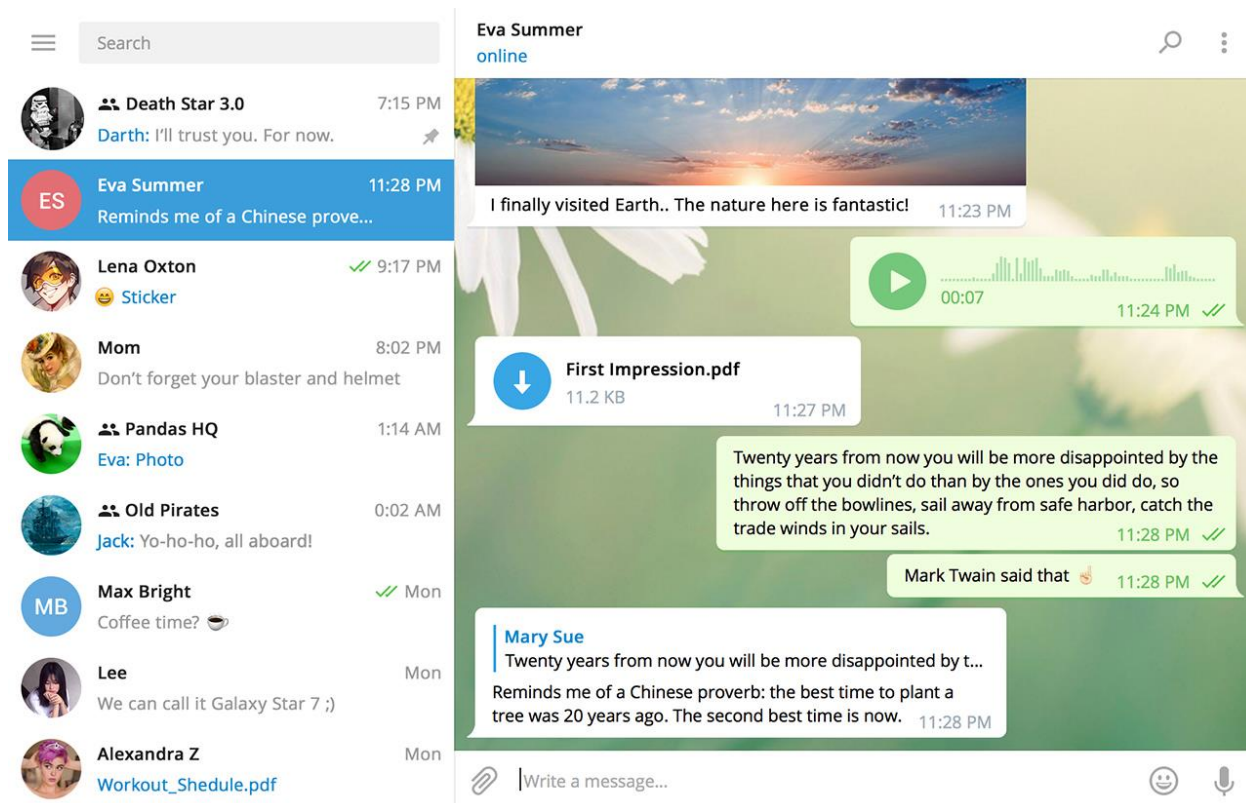


Рисунок 1.3 – Telegram Desktop

Проведённый анализ показал, что существующие решения обладают рядом общих недостатков, среди которых можно выделить зависимость от сторонних сервисов, невозможность полного контроля над данными, избыточный функционал и недостаточную гибкость настройки. Это подтверждает актуальность разработки собственного корпоративного мессенджера, который будет учитывать специфику внутреннего использования и обеспечивать необходимый уровень безопасности и управляемости.

1.2 Анализ предметной области

Корпоративный мессенджер представляет собой программную систему, предназначенную для организации обмена сообщениями между сотрудниками внутри одной организации. В отличие от публичных мессенджеров, такие системы ориентированы на решение задач, связанных с внутренними бизнес-процессами, и должны учитывать требования к безопасности, управлению доступом и хранению данных.

Основной функцией корпоративного мессенджера является обеспечение быстрого и удобного обмена информацией между пользователями. При этом система должна поддерживать как личные сообщения между двумя

сотрудниками, так и групповые чаты, предназначенные для коллективного обсуждения задач. Важной особенностью является возможность разграничения прав доступа, позволяющая определять роли пользователей и ограничивать доступ к определённым чатам или функциям системы.

Система должна обеспечивать хранение истории сообщений, что позволяет пользователям возвращаться к предыдущим обсуждениям и использовать их в дальнейшей работе. Кроме того, необходимо предусмотреть механизм поиска пользователей и чатов, что значительно упрощает навигацию и взаимодействие в системе.

Особое внимание должно уделяться безопасности данных. Это включает в себя защиту учетных записей пользователей, безопасное хранение паролей, а также использование защищенных каналов передачи данных. В корпоративной среде важно минимизировать риски утечки информации и обеспечить контроль над всеми данными, проходящими через систему.

С точки зрения архитектуры, корпоративный мессенджер представляет собой клиент-серверное приложение, в котором серверная часть отвечает за обработку запросов, хранение данных и управление логикой системы, а клиентская часть обеспечивает взаимодействие пользователя с системой через графический интерфейс. Обмен данными между клиентом и сервером осуществляется с использованием сетевых протоколов, а для обеспечения работы в реальном времени применяются технологии, позволяющие поддерживать постоянное соединение между сторонами.

Функционально систему можно разделить на несколько основных компонентов. К ним относятся модуль управления пользователями, обеспечивающий регистрацию, авторизацию и управление учетными записями, модуль обмена сообщениями, отвечающий за отправку, получение и отображение сообщений, а также модуль чатов, реализующий создание и управление диалогами и группами.

Таким образом, анализ предметной области позволяет сформулировать основные требования к разрабатываемому приложению и определить его ключевые особенности. Корпоративный мессенджер должен быть удобным, надежным, безопасным и масштабируемым решением, способным эффективно поддерживать коммуникацию внутри организации.

2 ТЕХНОЛОГИИ ПРОГРАММИРОВАНИЯ, ИСПОЛЬЗУЕМЫЕ ДЛЯ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ

2.1 Язык программирования и серверная платформа

Для разработки серверной части корпоративного мессенджера выбран язык программирования C# [2] и платформа ASP.NET Core [2]. Данный выбор обусловлен широкими возможностями, которые предоставляет экосистема .NET для создания современных веб-приложений, а также высокой производительностью и надежностью реализуемых решений.

Язык C# [2] является современным объектно-ориентированным языком программирования, обладающим строгой типизацией, развитой системой управления памятью и удобным синтаксисом. Он активно используется при разработке корпоративных систем различного уровня сложности и позволяет создавать масштабируемые и поддерживаемые приложения. Важным преимуществом C# является его интеграция с платформой .NET, которая предоставляет разработчику большое количество готовых инструментов и библиотек для решения типовых задач.

Платформа ASP.NET Core [2] представляет собой кроссплатформенный фреймворк для разработки веб-приложений и веб-сервисов. Она позволяет создавать как классические веб-приложения, так и Web API, обеспечивающие взаимодействие между серверной и клиентской частями системы. Одним из ключевых преимуществ ASP.NET Core является высокая производительность, достигаемая за счёт оптимизированной архитектуры и асинхронной обработки запросов. Кроме того, платформа поддерживает гибкую настройку маршрутизации, встроенные механизмы внедрения зависимостей и средства обеспечения безопасности.

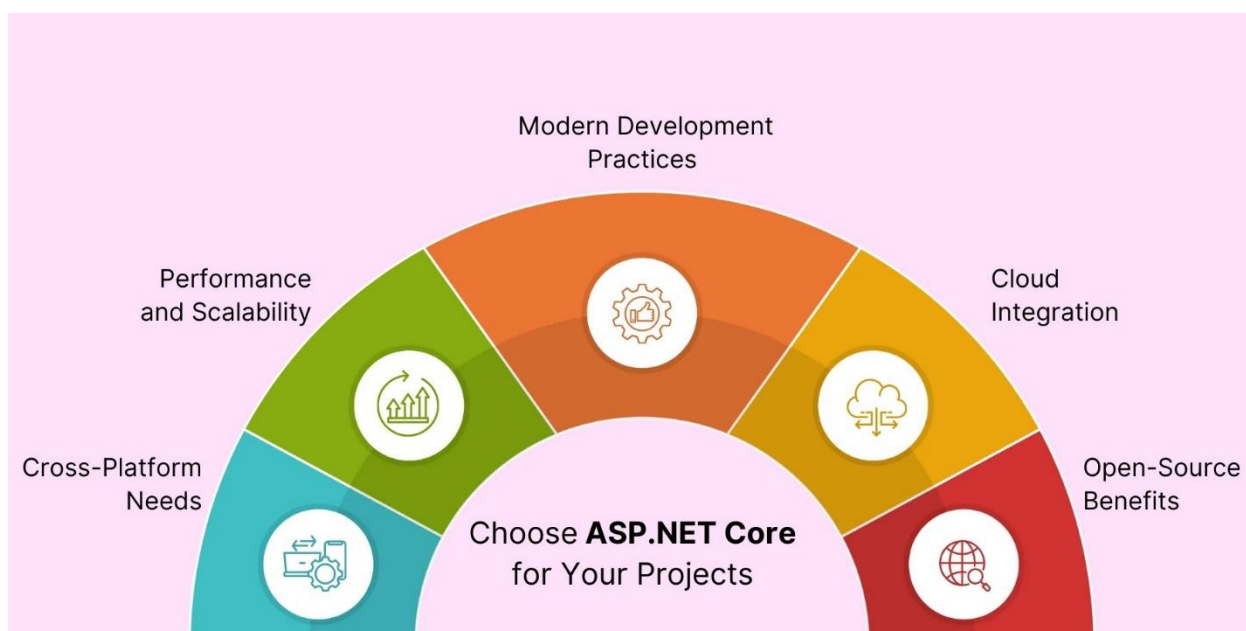


Рисунок 2.1 – Возможности платформы ASP.NET Core

В рамках разрабатываемого проекта серверная часть реализуется в виде Web API, что позволяет отделить логику обработки данных от пользовательского интерфейса. Такой подход соответствует современным требованиям к разработке веб-приложений и обеспечивает возможность использования различных технологий на стороне клиента. Сервер выполняет роль центрального компонента системы, обрабатывая запросы пользователей, управляя бизнес-логикой и обеспечивая взаимодействие с базой данных.

Использование ASP.NET Core также позволяет легко реализовать масштабирование системы, что является важным фактором для корпоративного мессенджера, рассчитанного на одновременную работу большого числа пользователей. Благодаря поддержке асинхронного программирования сервер способен эффективно обрабатывать множество параллельных запросов без существенного снижения производительности.

2.2 Технологии взаимодействия клиента и сервера

Взаимодействие между клиентской и серверной частями приложения осуществляется с использованием архитектурного подхода, основанного на передаче данных по сети с помощью HTTP-протокола. Для этого серверная часть предоставляет набор API-методов (эндпоинтов), которые позволяют клиенту выполнять операции, связанные с регистрацией пользователей, авторизацией, созданием чатов и отправкой сообщений.

Данные между клиентом и сервером передаются в формате JSON,

который является стандартом для обмена информацией в веб-приложениях. Этот формат отличается простотой, читаемостью и поддержкой во всех современных языках программирования, что делает его удобным для использования в распределённых системах.

Однако использование только HTTP-запросов недостаточно для реализации полноценного мессенджера, поскольку обмен сообщениями требует работы в реальном времени. В традиционной модели клиент отправляет запрос и получает ответ, после чего соединение закрывается. Такой подход не позволяет серверу самостоятельно инициировать отправку данных клиенту, что делает невозможной мгновенную доставку сообщений.

Для решения данной задачи используется технология постоянного соединения между клиентом и сервером, реализованная с помощью библиотеки SignalR [3]. Данная технология позволяет установить двусторонний канал связи, в рамках которого сервер может отправлять данные клиенту без необходимости постоянных запросов с его стороны. Это обеспечивает мгновенную доставку сообщений и делает взаимодействие пользователей максимально приближенным к реальному времени.

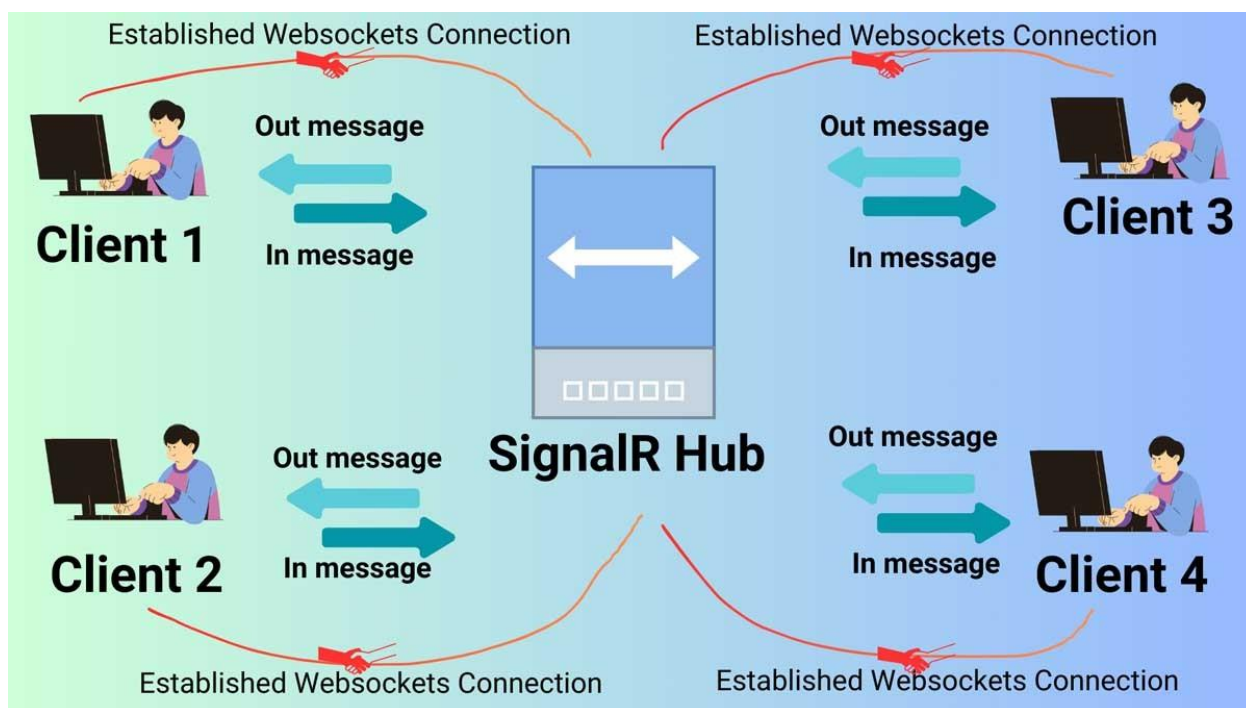


Рисунок 2.2 – Модель взаимодействия клиентов через SignalR Hub с использованием протокола WebSocket.

SignalR абстрагирует низкоуровневые механизмы работы с WebSocket [3] и автоматически выбирает наиболее подходящий способ передачи данных в зависимости от возможностей клиента и сервера. Это значительно упрощает разработку и позволяет сосредоточиться на логике приложения, не углубляясь

в детали сетевого взаимодействия.

Таким образом, в системе используется комбинированный подход: HTTP API применяется для выполнения стандартных операций, таких как регистрация и управление чатами, а SignalR — для обмена сообщениями в реальном времени.

2.3 База данных и работа с данными

Для хранения данных в проекте используется система управления базами данных PostgreSQL [4]. Данный выбор обусловлен высокой надежностью, производительностью и поддержкой сложных запросов, что делает PostgreSQL подходящей для разработки корпоративных систем.

База данных играет ключевую роль в работе мессенджера, поскольку в ней хранятся сведения о пользователях, чатах и сообщениях. Структура базы данных строится на основе реляционной модели, что обеспечивает целостность данных и возможность эффективного выполнения запросов.

Для взаимодействия с базой данных используется Entity Framework Core [5] — объектно-реляционное отображение (ORM), позволяющее работать с данными на уровне объектов. Это значительно упрощает разработку, так как избавляет от необходимости писать сложные SQL-запросы вручную и позволяет использовать возможности языка C# для управления данными.

Entity Framework Core поддерживает механизм миграций [5], который позволяет изменять структуру базы данных без потери данных. Это особенно важно в процессе разработки, когда структура приложения может изменяться и дополняться новыми сущностями.

Основными сущностями в базе данных являются пользователи, чаты и сообщения. Пользователь содержит информацию об учетной записи, чат определяет структуру общения и может быть личным, групповым или каналом, а сообщение представляет собой отдельную запись, связанную с конкретным чатом и отправителем, что позволяет хранить историю переписки.

2.4 Архитектура программного обеспечения

В качестве архитектурного подхода для разработки серверной части приложения используется Clean Architecture [6]. Такой подход позволяет разделить систему на логические уровни, каждый из которых отвечает за определённую часть функциональности.

Основная идея данной архитектуры заключается в разделении

ответственности между слоями, что делает систему более гибкой, тестируемой и удобной для сопровождения. Внутренние слои не зависят от внешних, что позволяет изменять детали реализации без затрагивания бизнес-логики.

Уровень представления реализован через Web API и отвечает за обработку входящих HTTP-запросов от клиента, их валидацию и формирование ответов. Контроллеры выступают точкой входа в систему и не содержат бизнес-логики, а лишь делегируют выполнение операций на нижележащие уровни.

Уровень приложения (Application) реализует основную бизнес-логику системы и построен с использованием паттерна CQRS [7]. Для организации взаимодействия между компонентами применяется библиотека MediatR, которая позволяет разделить команды (изменяющие состояние) и запросы (чтение данных), а также обеспечить слабую связанность между обработчиками и вызывающим кодом.

Уровень инфраструктуры (Infrastructure) отвечает за работу с внешними зависимостями, в частности с базой данных. Доступ к данным организован с использованием паттерна Repository, который инкапсулирует операции чтения и записи, а также паттерна Unit of Work, обеспечивающего управление транзакциями и согласованность изменений. Репозитории и Unit of Work совместно используются обработчиками команд и запросов из слоя Application.

Таким образом, взаимодействие между слоями построено через четко определённые абстракции: контроллеры передают запросы в слой Application через MediatR [8], обработчики используют Unit of Work и репозитории из слоя Infrastructure, что обеспечивает модульность, тестируемость и расширяемость системы.

Использование внедрения зависимостей позволяет снизить связанность компонентов и упростить тестирование системы. Все зависимости передаются через конструкторы классов, что делает код более прозрачным и управляемым.

Clean Architecture Dependencies

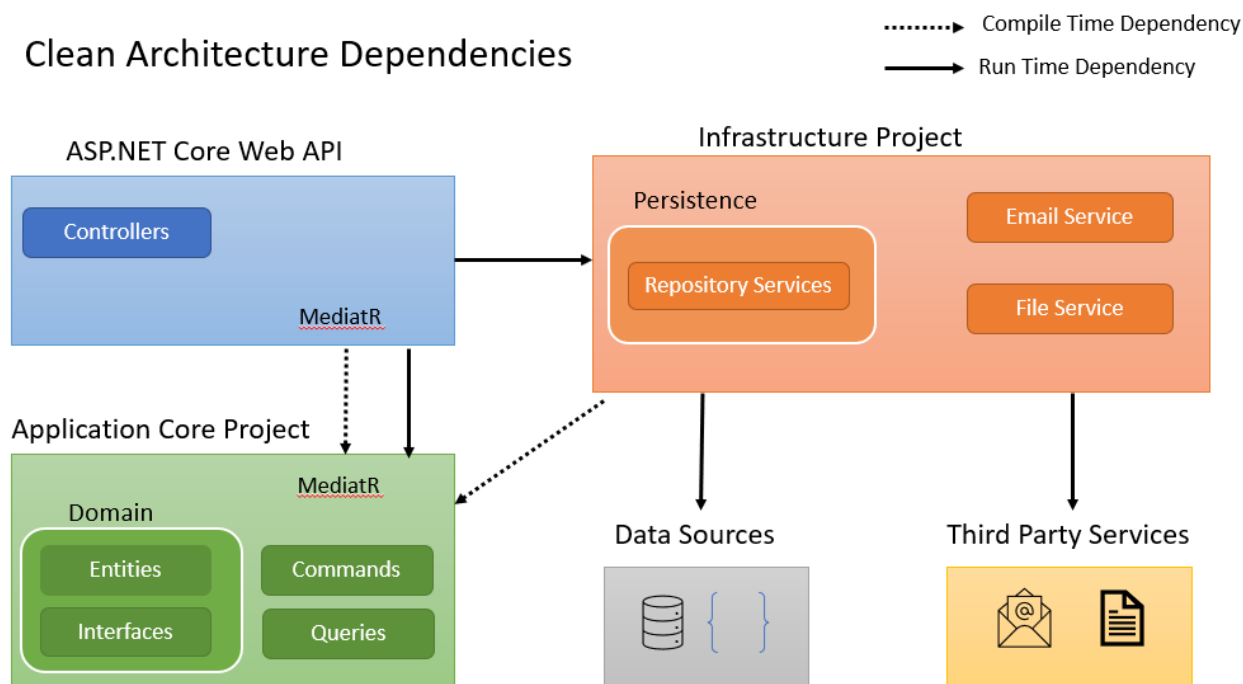


Рисунок 2.3 – иллюстрация чистой архитектуры в контексте ASP.NET Core

2.5 Клиентская часть приложения

Клиентская часть корпоративного мессенджера реализована в виде одностраничного веб-приложения на базе библиотеки React [9]. Выбор данной технологии обусловлен её компонентно-ориентированным подходом, который позволяет эффективно разделять интерфейс на независимые логические модули, упрощая поддержку и масштабирование системы.

Для обеспечения строгой типизации данных и повышения надежности кода используется язык программирования TypeScript, что минимизирует вероятность возникновения ошибок на этапе выполнения и упрощает проектирование сложных интерфейсов.

Управление состоянием приложения организовано с помощью библиотеки Zustand, которая предоставляет производительный и предсказуемый механизм распределения данных между компонентами.

Внешний вид приложения реализован с использованием фреймворка Tailwind CSS, обеспечивающего гибкую настройку стилей и адаптивность интерфейса под различные устройства.

Взаимодействие с серверной частью осуществляется посредством протокола HTTP с применением библиотеки Axios для выполнения асинхронных запросов. Такой технологический стек в совокупности с современными инструментами сборки позволяет создать отзывчивое и производительное приложение, обеспечивающее бесшовное взаимодействие

пользователя с системой обмена сообщениями без необходимости полной перезагрузки страниц.

2.6 Обеспечение безопасности

Безопасность является одним из приоритетных аспектов разработки корпоративного мессенджера, так как система предназначена для обработки конфиденциальных данных и внутренних коммуникаций организации.

В рамках проекта реализована комплексная система защиты, основанная на интеграции встроенного механизма ASP.NET Core Identity [10] и технологии JSON Web Tokens (JWT). Использование инфраструктуры Identity позволяет эффективно организовать процессы регистрации и управления учетными записями, обеспечивая при этом высокий уровень защиты паролей, которые хранятся исключительно в виде криптографических хэшей.

Для обеспечения безопасности клиент-серверного взаимодействия и реализации концепции Stateless-архитектуры применена аутентификация на основе JWT-токенов. Данный подход подразумевает выдачу зашифрованного токена после успешного входа в систему, который используется для идентификации пользователя при каждом последующем запросе. Это исключает необходимость хранения сессий на сервере и повышает устойчивость системы к нагрузкам. Дополнительно реализована система разграничения прав доступа на основе ролей, встроенная в конвейер обработки запросов ASP.NET Core, что позволяет гибко ограничивать доступ к функциональным возможностям мессенджера в зависимости от полномочий конкретного пользователя. Внедрение данных технологий в совокупности обеспечивает надежную защиту от несанкционированного доступа и гарантирует целостность и конфиденциальность передаваемой информации в рамках корпоративной среды.

3 РАЗРАБОТКА ПРОГРАММНОГО ПРИЛОЖЕНИЯ

3.1 Общая архитектура системы

В рамках практической реализации разработанного проекта была спроектирована и реализована клиент-серверная архитектура приложения.

Серверная часть приложения реализована с использованием технологии ASP.NET Core [2] и представляет собой Web API, отвечающий за обработку входящих запросов, выполнение бизнес-логики и взаимодействие с базой данных. Сервер выступает центральным элементом системы, обеспечивая управление пользователями, чатами и сообщениями, а также реализуя механизмы аутентификации, авторизации и обеспечения безопасности. Вся логика обработки данных сосредоточена на сервере, что позволяет минимизировать нагрузку на клиентскую часть и обеспечить целостность и контроль над данными.

Клиентская часть реализована в виде одностраничного веб-приложения с использованием библиотеки React [9]. Данный подход позволяет обеспечить высокую отзывчивость интерфейса и комфортное взаимодействие пользователя с системой без необходимости полной перезагрузки страниц. Клиент отвечает за отображение данных, обработку пользовательских действий и формирование запросов к серверу, а также за установление соединения для обмена сообщениями в реальном времени.

Взаимодействие между клиентом и сервером организовано с использованием протокола HTTP, который применяется для выполнения стандартных операций, таких как регистрация пользователей, авторизация, получение списка чатов и управление сообщениями. Передача данных осуществляется в формате JSON, что обеспечивает универсальность и удобство обработки информации на обеих сторонах приложения.

Для реализации обмена сообщениями в реальном времени используется технология SignalR [3], позволяющая установить постоянное двустороннее соединение между клиентом и сервером. В отличие от классической модели взаимодействия по HTTP, данный подход позволяет серверу самостоятельно инициировать отправку данных клиенту, что обеспечивает мгновенную доставку сообщений и актуализацию данных без необходимости периодических запросов. Это особенно важно для мессенджера, где критическим требованием является минимальная задержка при передаче сообщений.

Структура программного комплекса корпоративного мессенджера

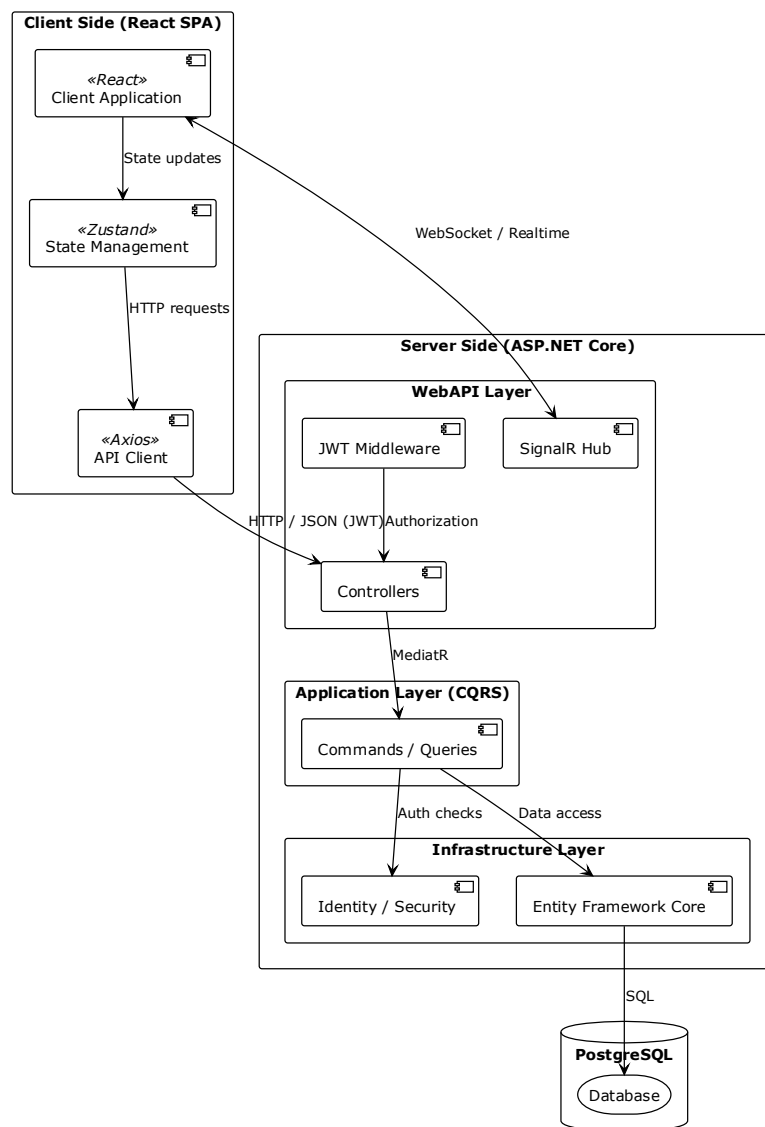


Рисунок 3.1 – Структура программного комплекса корпоративного мессенджера

3.2 Архитектура серверной части

3.2.1 Структура проекта

Слой домена является ядром программного комплекса и содержит в себе описание фундаментальной бизнес-логики и сущностей предметной области мессенджера. Центральное место в данном слое занимают сущности, такие как Chat, Message, User и ChatMember, которые определяют структуру данных и

правила их взаимодействия, не завися от внешних фреймворков, баз данных или сторонних библиотек. Использование базового класса BaseEntity позволяет унифицировать идентификацию всех объектов системы, а перечисления в папке Enums строго регламентируют возможные состояния системы, например, типы чатов или роли пользователей, обеспечивая целостность бизнес-правил на самом низком уровне.

Помимо моделей данных, доменный слой определяет интерфейсы репозитория и паттерна Unit of Work, которые задают контракты для работы с данными, но не содержат их технической реализации. Это соответствует принципам чистой архитектуры, согласно которым зависимости всегда направлены вовнутрь: домен не знает о существовании базы данных или API, он лишь диктует условия, которым должны соответствовать внешние слои. Такое решение обеспечивает высокую тестируемость системы и позволяет изменять способ хранения данных в будущем без необходимости корректировки основной логики приложения.

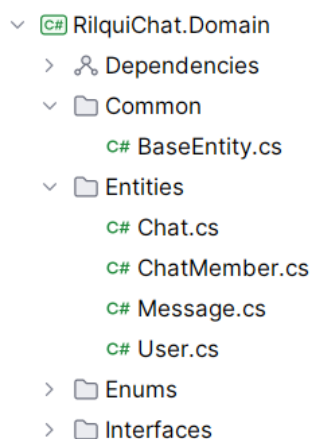


Рисунок 3.2 – Структура доменного слоя

Слой приложения (Application) отвечает за реализацию прикладной логики системы и координирует выполнение пользовательских сценариев. В основе данного слоя лежит паттерн CQRS, который разделяет операции изменения данных (Commands) и операции их чтения (Queries). Это позволяет структурировать логику мессенджера по функциональным возможностям в папке Features, таким как управление чатами, сообщениями и профилями пользователей. Каждая функция инкапсулирует в себе не только саму логику обработки, но и правила валидации входных данных, что гарантирует корректность выполнения операций до их перехода на уровень доступа к данным.

Важной составляющей этого слоя является использование механизма

Pipeline Behaviors программной библиотеки MediatR. Это позволяет вынести сквозную функциональность, такую как автоматическая валидация запросов и управление транзакциями базы данных, в отдельные обработчики (ValidationBehavior и TransactionBehavior). Таким образом, основной код бизнес-логики остается «чистым» и сфокусированным только на выполнении конкретных задач. Также здесь определяются объекты передачи данных (DTO), которые служат контрактами для взаимодействия с внешним миром, предотвращая утечку внутренних доменных моделей за пределы слоя.

Слой приложения полностью абстрагирован от технических деталей реализации внешних сервисов через использование интерфейсов. В папке Common/Interfaces определены контракты для работы с аутентификацией, генерацией JWT-токенов, хранением файлов и рассылкой уведомлений через SignalR. Сама реализация этих интерфейсов вынесена в слой инфраструктуры, что полностью соответствует принципу инверсии зависимостей и позволяет легко заменять технические компоненты системы без изменения логики работы мессенджера.

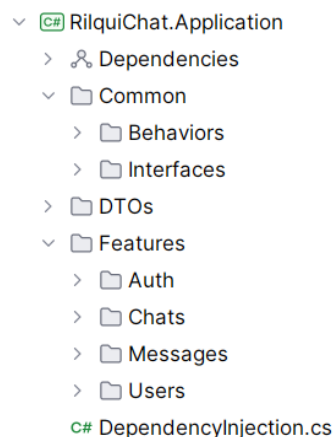


Рисунок 3.3 – Структура слоя приложения

Слой инфраструктуры (Infrastructure) содержит техническую реализацию всех внешних интерфейсов, определенных в слоях домена и приложения, обеспечивая взаимодействие системы с базой данных и сторонними сервисами. Основу этого слоя составляет работа с данными через Entity Framework Core, где в классе AppDbContext настроено отображение доменных сущностей на таблицы базы данных. Использование Fluent API в папке Configurations позволяет детально настроить схему базы данных, сохраняя при этом чистоту самих доменных моделей.

В данном слое реализован паттерн Repository (Репозиторий) и Unit of Work (Единица работы), которые предоставляют конкретные механизмы

доступа к данным и управления транзакциями. Базовый класс `RepositoryBase` инкапсулирует общие методы работы с контекстом БД, что исключает дублирование кода и упрощает поддержку проекта.

Кроме работы с данными, слой инфраструктуры отвечает за безопасность и системные сервисы. Здесь реализована интеграция с ASP.NET Core Identity для управления учетными записями, а также конкретные механизмы генерации JWT-токенов в сервисе `JwtTokenGenerator`. Сервисы в соответствующей папке решают прикладные задачи, такие как определение текущего пользователя через `CurrentUserService` или работа с файловой системой в `LocalFileStorageService`. Такая организация позволяет скрыть детали реализации от бизнес-логики, строго следуя принципам чистой архитектуры.

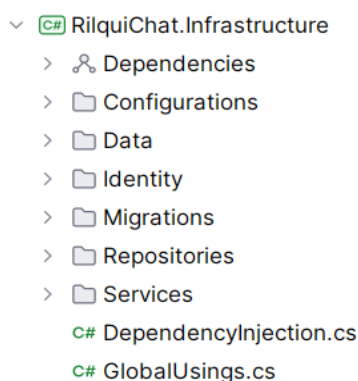


Рисунок 3.4 – Структура слоя инфраструктуры

Слой веб-API (WebAPI) является точкой входа в приложение и отвечает за обработку входящих HTTP-запросов, а также организацию двустороннего обмена данными в реальном времени. В данном слое реализованы контроллеры, которые наследуются от базового класса `BaseApiController`. Их основной задачей является делегирование обработки запросов слою приложения через библиотеку `MediatR`, что позволяет контроллерам содержать только логику маршрутизации и возврата HTTP-ответов. Использование атрибутов аутентификации в контроллерах обеспечивает защиту конечных точек и гарантирует доступ к функционалу только авторизованным пользователям.

Особое место в реализации мессенджера занимает папка `Hubs`, где находится `ChatHub` — центральный узел для работы с технологией `SignalR`. Этот компонент обеспечивает мгновенную доставку сообщений и уведомлений подключенным клиентам. Сопутствующий сервис `SignalRService` выступает связующим звеном, позволяя отправлять

уведомления пользователям напрямую из обработчиков бизнес-логики, не нарушая при этом границы слоев архитектуры.

Конфигурация всего приложения сосредоточена в файле Program.cs, где происходит регистрация сервисов в контейнере зависимостей и настройка конвейера обработки запросов (Middleware), включая аутентификацию, авторизацию и поддержку CORS. Наличие статической папки wwwroot/uploads позволяет системе обслуживать загруженные пользователями файлы, такие как аватары или вложения. Таким образом, слой WebAPI объединяет все части системы в единый работающий программный комплекс.

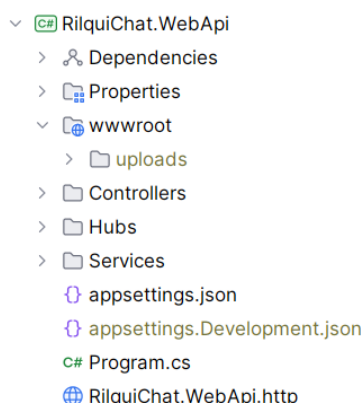


Рисунок 3.5 – Структура слоя веб-API

3.2.2 Реализация паттернов проектирования

В ходе разработки архитектуры серверной части были применены фундаментальные паттерны проектирования, обеспечивающие модульность, тестируемость и слабую связанность компонентов системы. Основу архитектурного взаимодействия составляет паттерн Mediator, реализованный с помощью библиотеки MediatR [8]. Он позволяет инкапсулировать логику обработки запросов в отдельных обработчиках, исключая прямые зависимости между слоем API и бизнес-логикой. Это неразрывно связано с паттерном CQRS (Command Query Responsibility Segregation) [7], который разделяет модель системы на две части: команды для изменения состояния и запросы для получения данных, что оптимизирует производительность и упрощает поддержку каждой из моделей.

Для управления доступом к данным и абстрагирования от деталей реализации ORM (Entity Framework Core) применены паттерны Repository (Репозиторий) и Unit of Work (Единица работы) [11]. Репозитории

предоставляют коллекции для работы с доменными сущностями, скрывая сложность SQL-запросов, а Unit of Work гарантирует атомарность операций, позволяя выполнить группу действий в рамках единой транзакции базы данных. Такой подход обеспечивает целостность информации при выполнении сложных бизнес-сценариев, таких как создание группового чата с одновременным добавлением нескольких участников.

На системном уровне ключевую роль играет паттерн Dependency Injection (Внедрение зависимостей), встроенный в платформу ASP.NET Core. Он используется для управления жизненным циклом сервисов и обеспечения инверсии управления, что позволяет подставлять различные реализации интерфейсов (например, для локального хранения файлов или облачного) без изменения потребителей. Также в проекте реализован паттерн Proxy и Decorator через механизмы MediatR Behaviors, что позволило внедрить сквозную логику валидации, не загромождая основные обработчики команд. Для преобразования данных между слоями (из сущностей базы данных в объекты передачи данных DTO) применен паттерн Data Transfer Object, минимизирующий объем передаваемой по сети информации и скрывающий внутреннюю структуру базы данных от клиента.

На примере сценария отправки сообщения в чате рассмотрим, как все перечисленные паттерны работают в единой связке.

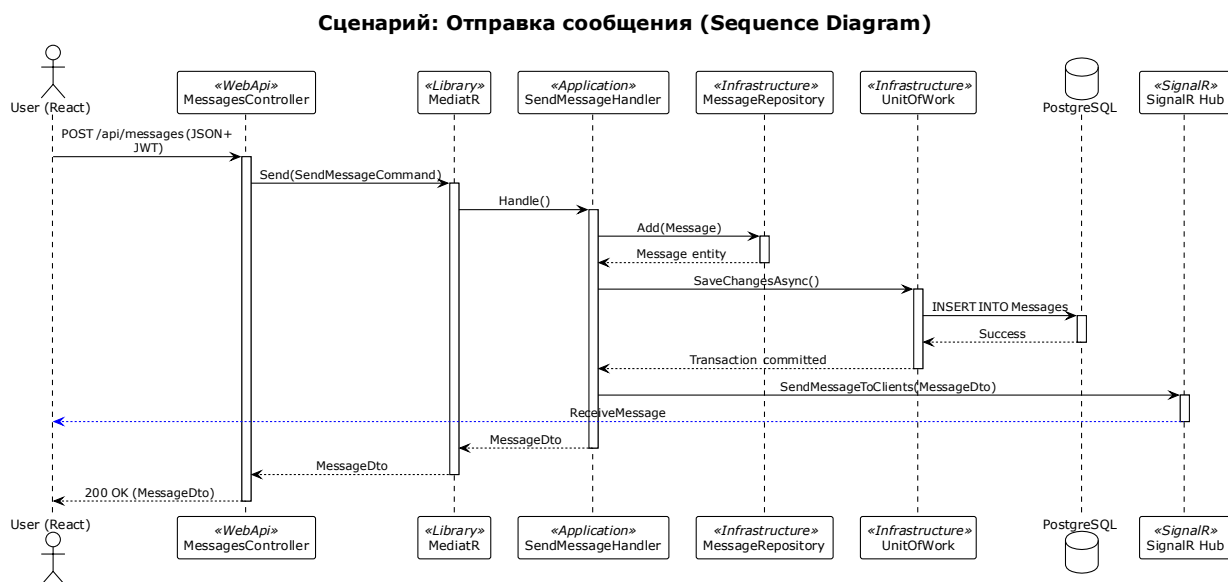


Рисунок 3.6 – диаграмма последовательности для отправки сообщения

Представленная диаграмма последовательности иллюстрирует полный цикл обработки запроса на отправку сообщения, начиная от действия пользователя в интерфейсе и заканчивая обновлением данных у всех

участников чата.

Процесс инициируется клиентской частью, которая отправляет HTTP-запрос на сервер, содержащий текст сообщения и данные для аутентификации. На стороне сервера контроллер принимает запрос и передает задачу в шину данных MediatR. Внутренний механизм шины находит соответствующий обработчик (Handler), который берет на себя выполнение основной бизнес-логики.

В ходе выполнения обработчик взаимодействует с репозиторием для создания новой сущности сообщения и использует паттерн Unit of Work для фиксации изменений в базе данных. Это гарантирует, что сообщение будет сохранено только в случае успешного завершения всей транзакции. После подтверждения от базы данных обработчик обращается к SignalR Hub для мгновенной рассылки уведомления всем активным участникам диалога. Завершается процесс возвратом сформированного объекта данных (DTO) обратно контроллеру и отправкой успешного статуса ответа пользователю, что обеспечивает плавный и отзывчивый интерфейс мессенджера.

3.2.3 Реализация API (эндпоинты)

Взаимодействие между клиентской и серверной частями мессенджера организовано посредством REST-ориентированного программного интерфейса. Проектирование эндпоинтов выполнено с соблюдением принципов семантики HTTP-методов, где GET используется для получения данных, POST — для создания новых ресурсов, PUT и PATCH — для полной или частичной модификации, а DELETE — для их удаления. Все данные передаются в формате JSON, что обеспечивает легкость интеграции и высокую скорость парсинга на стороне клиента.

Для обеспечения безопасности большинство конечных точек защищены механизмом авторизации. Доступ к ним возможен только при наличии в заголовке запроса (Authorization) валидного JWT-токена, выданного сервером при успешном входе в систему. Исключение составляют эндпоинты регистрации и аутентификации, которые находятся в открытом доступе

Метод	URL	Описание
POST	/api/user/register	Регистрация нового пользователя
POST	/api/user/login	Аутентификация пользователя и получение JWT-токена

GET	/api/user/me	Получение информации о текущем пользователе
PUT	/api/user/profile	Обновление профиля пользователя
GET	/api/user/search	Поиск пользователей по имени
POST	/api/user/avatar	Загрузка аватара пользователя
GET	/api/chats	Получение списка чатов пользователя
GET	/api/chats/{id}	Получение детальной информации о чате
POST	/api/chats/direct	Создание личного чата
POST	/api/chats/group	Создание группового чата или канала
POST	/api/chats/{id}/members	Добавление пользователя в чат
DELETE	/api/chats/{id}/members/{userId}	Удаление пользователя из чата
PATCH	/api/chats/{id}/rename	Переименование чата
POST	/api/chats/{id}/read	Пометка сообщений как прочитанных
GET	/api/chats/{id}/pinned	Получение закреплённых сообщений
GET	/api/messages/{chatId}	Получение истории сообщений
POST	/api/messages	Отправка текстового сообщения
POST	/api/messages/file	Отправка файла
PUT	/api/messages/{id}	Редактирование сообщения
DELETE	/api/messages/{id}	Удаление сообщения
PATCH	/api/messages/{id}/pin	Закрепление сообщения
PATCH	/api/messages/{id}/unpin	Открепление сообщения
GET	/api/messages/{chatId}/search	Поиск сообщений в чате

Таблица 3.1 – эндпоинты сервера

Для документирования и тестирования реализованных эндпоинтов в проект интегрирован инструмент Swagger UI. Он предоставляет интерактивную графическую оболочку, позволяющую разработчику просматривать структуру схем данных (DTO) и выполнять пробные запросы к API в реальном времени.

3.2.4 Модель базы данных

Для проектирования системы хранения данных мессенджера была выбрана реляционная модель, обеспечивающая высокую целостность информации и эффективное управление связями между участниками коммуникации. Логическая структура базы данных организована таким образом, чтобы поддерживать как хранение метаданных пользователей, так и сложную иерархию сообщений внутри различных типов чатов. Представленная ниже ER-диаграмма отображает ключевые сущности системы и их взаимодействие в рамках архитектуры программного комплекса.

ER-диаграмма базы данных корпоративного мессенджера

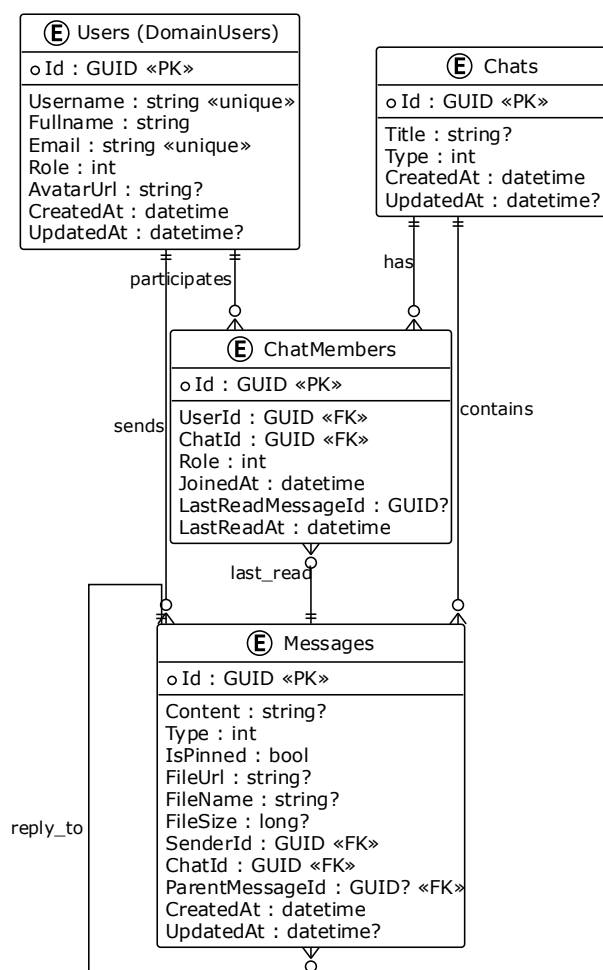


Рисунок 3.7 – ER-диаграмма базы данных

Центральным элементом архитектуры данных является нормализованная структура связей, где взаимодействие между пользователями и чатами реализуется через промежуточную сущность

участников. Это позволяет не только гибко управлять доступом к переписке, но и отслеживать специфические состояния для каждого пользователя.

Модель сообщений спроектирована с учетом поддержки мультимедийного контента и древовидных обсуждений. Использование рекурсивной связи внутри таблицы сообщений позволяет организовать систему ответов, где каждое сообщение может ссылаться на родительский объект. При этом связи с сущностями отправителя и чата обеспечивают быструю выборку истории переписки с сохранением авторства

3.3 Реализация клиентской части

3.3.1 Общая структура фронтенда

Клиентская часть мессенджера реализована как одностраничное веб-приложение (SPA) на базе библиотеки React [9] с использованием TypeScript. Использование строгой типизации на фронтенде позволяет синхронизировать структуры данных с серверной частью, что значительно снижает количество ошибок при обработке ответов от API.

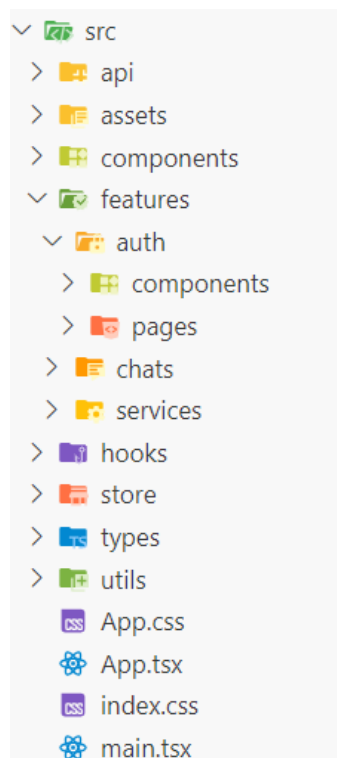


Рисунок 3.8 – Структура клиентской части

Централизованное управление данными в приложении реализовано с

помощью библиотеки Zustand. Это решение позволило создать легковесные и производительные хранилища для управления авторизацией пользователя и состоянием активных чатов, обеспечивая мгновенную реакцию интерфейса на изменения данных без избыточных перерисовок компонентов. Для стилизации интерфейса применен фреймворк Tailwind CSS, который обеспечивает адаптивность мессенджера под различные разрешения экранов и консистентность визуального оформления.

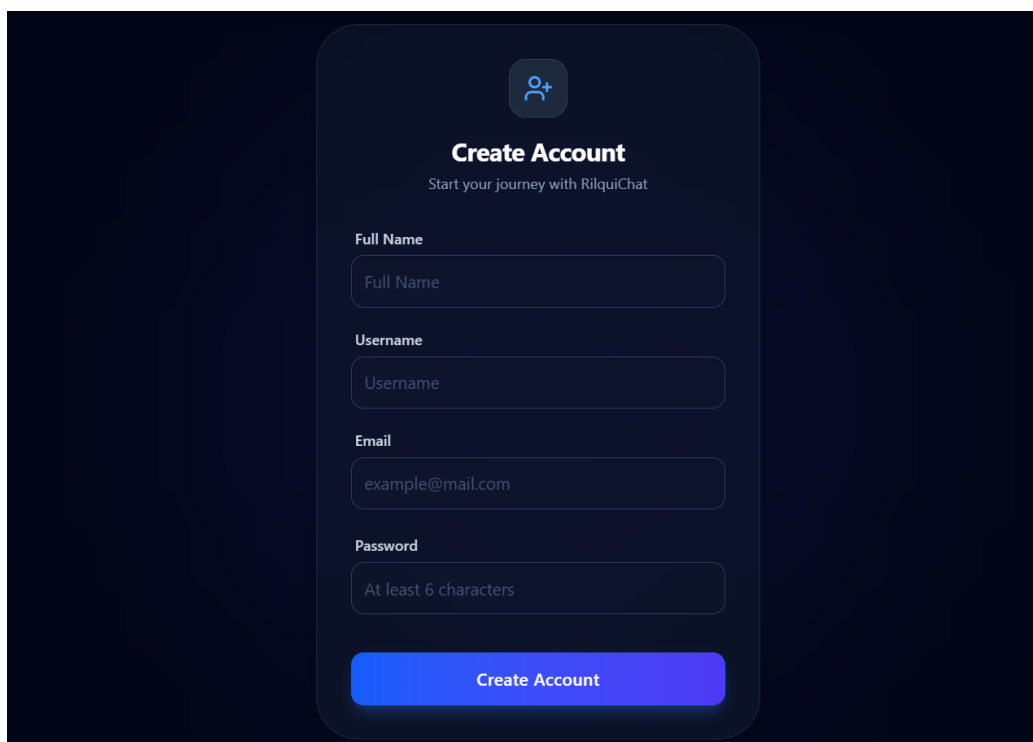
Взаимодействие с серверной частью организовано через два дополняющих друг друга механизма. Основной обмен данными, такой как авторизация, получение истории сообщений и загрузка файлов, осуществляется посредством асинхронных HTTP-запросов с использованием библиотеки Axios. Для обеспечения безопасности в каждый запрос автоматически внедряется JWT-токен через механизмы перехватчиков (interceptors). В свою очередь, для реализации функций реального времени используется клиент SignalR [3]. Он поддерживает постоянное соединение с сервером, позволяя мгновенно отображать входящие сообщения, уведомления о наборе текста и статусы прочтения без необходимости повторного опроса сервера клиентом. Такой гибридный подход обеспечивает высокую отзывчивость системы и соответствует стандартам современных корпоративных инструментов связи.

3.3.2 Пользовательский интерфейс

Процесс взаимодействия с программным комплексом начинается с этапа идентификации и аутентификации пользователя. Для обеспечения доступа к функциям мессенджера реализованы интуитивно понятные формы входа и создания новой учетной записи. При первом посещении приложения пользователю предлагается пройти процедуру регистрации, в ходе которой осуществляется сбор необходимых данных: полного имени, уникального псевдонима (username) и адреса электронной почты. На данном этапе внедрены механизмы предварительной валидации, предотвращающие отправку пустых полей или некорректных адресов, что снижает нагрузку на серверную часть и ускоряет процесс регистрации.

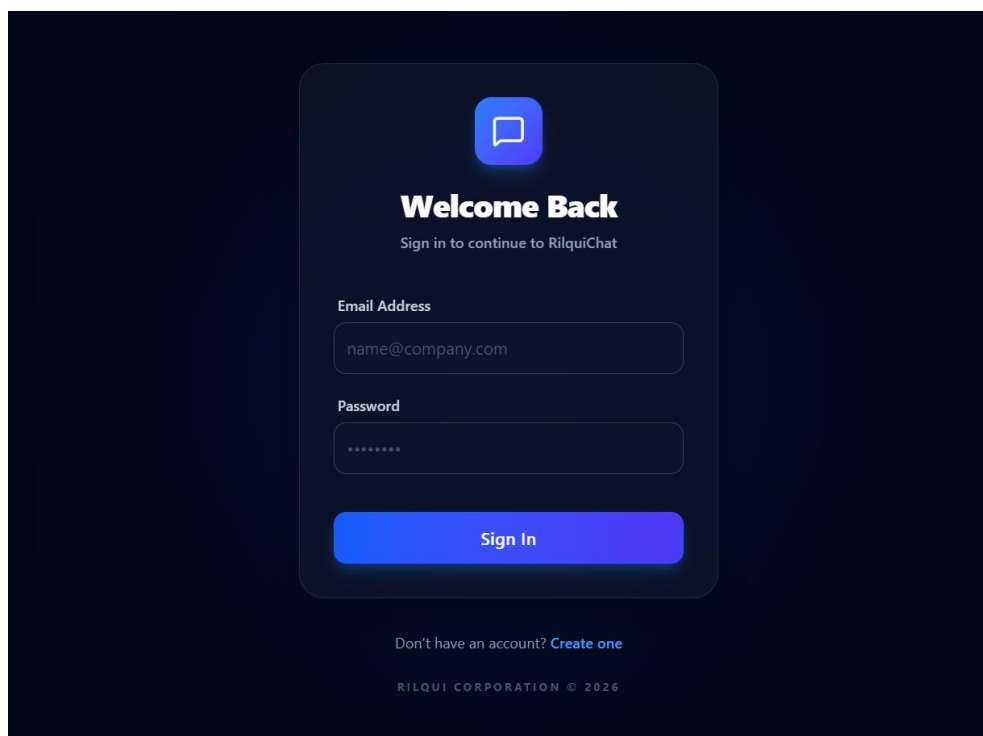
После создания учетной записи или при повторном входе в систему пользователь перенаправляется на форму аутентификации. Взаимодействие с данной формой инициирует процесс обмена данными с сервером для получения JWT-токена, который в дальнейшем используется для авторизации всех последующих запросов. Интерфейс этих страниц спроектирован таким образом, чтобы минимизировать когнитивную нагрузку, предлагая

пользователю сфокусироваться на вводе учетных данных в центре экрана.



The registration window features a dark blue background with a central white rounded rectangle. At the top of the rectangle is a blue icon of a person with a plus sign. Below the icon, the text "Create Account" is displayed in bold, followed by the subtitle "Start your journey with RilquiChat". The form contains four input fields: "Full Name" (placeholder: Full Name), "Username" (placeholder: Username), "Email" (placeholder: example@mail.com), and "Password" (placeholder: At least 6 characters). A blue "Create Account" button is positioned at the bottom of the form.

Рисунок 3.9 – Окно регистрации



The login window features a dark blue background with a central white rounded rectangle. At the top of the rectangle is a blue icon of a speech bubble. Below the icon, the text "Welcome Back" is displayed in bold, followed by the subtitle "Sign in to continue to RilquiChat". The form contains two input fields: "Email Address" (placeholder: name@company.com) and "Password" (placeholder: *****). A blue "Sign In" button is positioned at the bottom of the form. Below the button, there is a link: "Don't have an account? [Create one](#)". At the very bottom of the screen, the text "RILQUI CORPORATION © 2026" is displayed.

Рисунок 3.10 – Окно авторизации

После успешной аутентификации пользователь попадает в основное рабочее пространство приложения. Интерфейс главной страницы разделен на две функциональные зоны: боковую панель (Sidebar) для управления списком диалогов и основную область чата (ChatWindow). Такое разделение позволяет сохранять контекст общения, обеспечивая при этом быстрый доступ к навигации.

Боковая панель включает в себя инструменты для глобального поиска пользователей и фильтрации активных диалогов по категориям (личные сообщения, группы). Для удобства администрирования здесь же расположены кнопки быстрого создания новых чатов и перехода к настройкам профиля. Список чатов динамически обновляется: для каждого элемента отображается аватар собеседника, заголовок, последнее сообщение и индикация непрочитанных уведомлений.

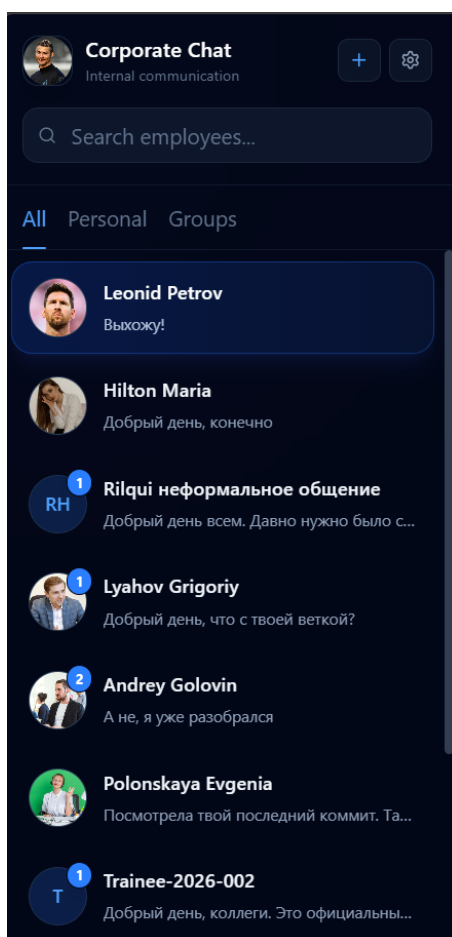


Рисунок 3.11 – Боковая панель

Центральную часть интерфейса занимает окно активного диалога. В верхней части окна (Header) отображается текущий статус собеседника и инструменты управления конкретным чатом, включая поиск по истории

сообщений и панель детальной информации. Одной из ключевых особенностей интерфейса является закрепленная область для важных сообщений (Pinned Messages), расположенная непосредственно под заголовком чата.

Основная область сообщений поддерживает бесконечную прокрутку с автоматической подгрузкой истории. Сообщения визуально разделены на входящие и исходящие, снабжены временными метками и индикаторами редактирования. Нижняя часть окна содержит поле ввода с поддержкой многострочного текста. В случае отсутствия прав на отправку сообщений (например, в информационных каналах) поле ввода автоматически заменяется на соответствующее уведомление, информирующее пользователя об ограничениях.

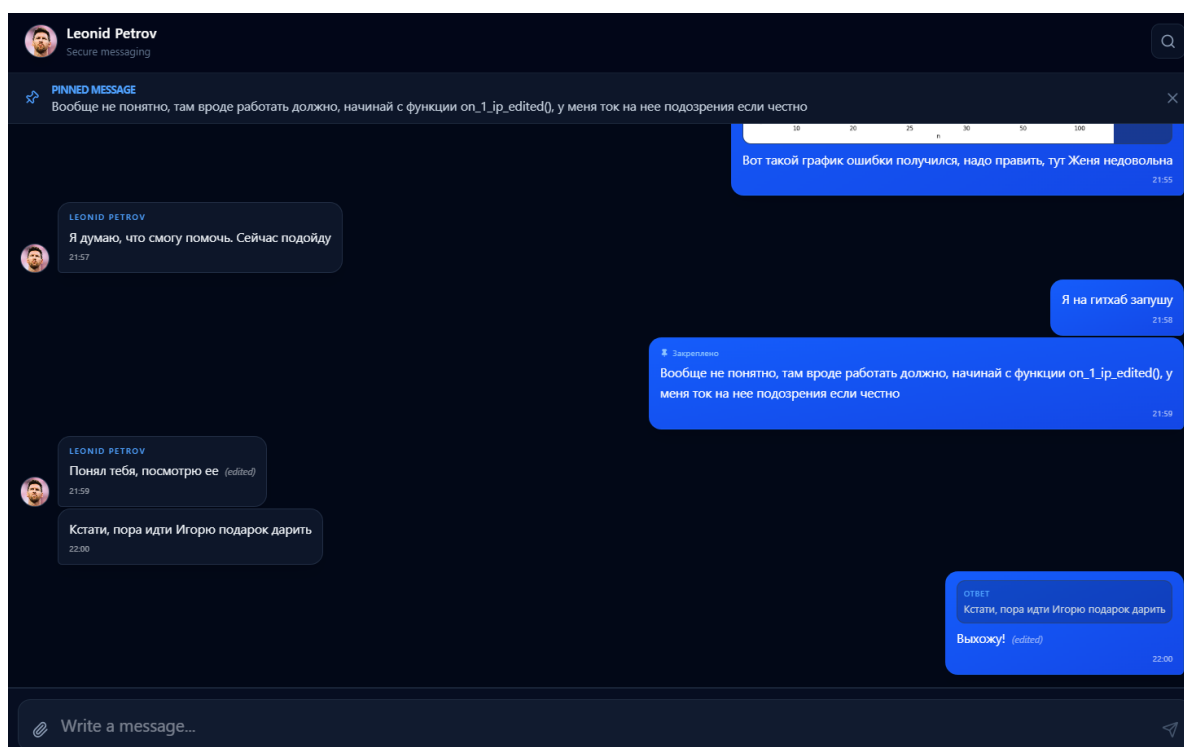


Рисунок 3.12 – Окно чата

Для расширения круга общения и создания тематических сообществ в приложении реализован функционал создания новых чатов через специализированное модальное окно. Данный компонент позволяет пользователю выбрать один из двух типов сущностей: приватную группу для совместного обсуждения или информационный канал с ограниченными правами публикации.

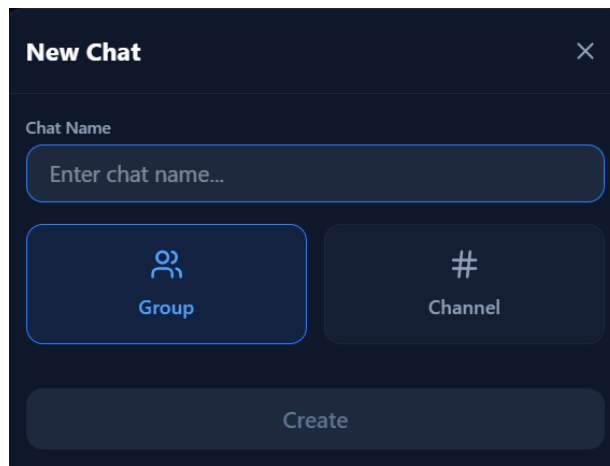


Рисунок 3.13 – Окно создания чата

Персонализация учетной записи осуществляется через интерфейс настроек профиля. Данный раздел позволяет пользователю актуализировать свои персональные данные: изменять отображаемое имя, управлять уникальным идентификатором пользователя и изменять аватар.

Технически форма редактирования тесно интегрирована с сервисом управления состоянием. При внесении изменений данные отправляются на сервер через соответствующий эндпоинт API, после чего глобальный объект пользователя в хранилище приложения синхронизируется с обновленным ответом от сервера. Это позволяет избежать несоответствия данных: например, новое имя пользователя мгновенно отображается во всех активных чатах и сайдбаре без необходимости перезагрузки страницы.

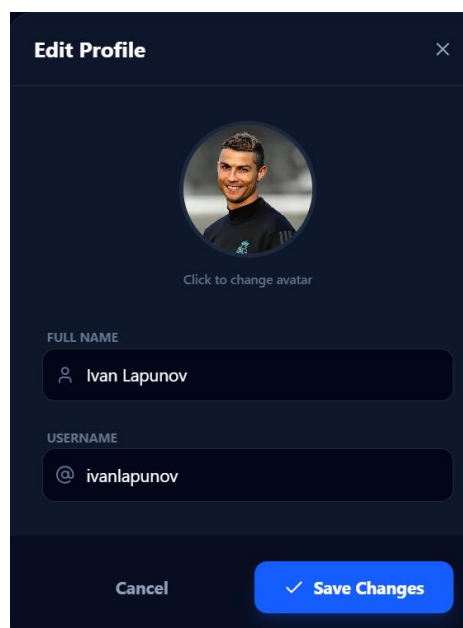


Рисунок 3.14 – Окно изменения профиля

4 ТЕСТИРОВАНИЕ

Тестирование является важным этапом разработки программного обеспечения, поскольку позволяет убедиться в корректности работы системы, своевременно выявить ошибки и повысить надежность приложения.

В рамках данной курсовой работы для автоматизированного тестирования использовалась технология xUnit [12], предназначенная для написания unit-тестов в среде .NET. Дополнительно применялись библиотеки Moq для создания mock-объектов и FluentAssertions для более удобной и читаемой проверки результатов тестирования.

В ходе работы были протестированы доменные сущности системы, включая пользователей, чаты, сообщения и участников чатов, что позволило проверить корректность бизнес-логики, валидации данных и обработки исключительных ситуаций. Также были протестированы use cases (handlers), реализующие основные сценарии работы приложения: авторизацию и регистрацию пользователей, отправку сообщений и файлов, управление чатами, изменение профиля пользователя и получение данных. Тестирование охватывало как успешные сценарии выполнения операций, так и обработку ошибочных ситуаций, включая некорректные входные данные, дублирование сущностей и ограничения бизнес-логики. Полученные результаты подтвердили корректность работы реализованной системы и устойчивость приложения к различным типам ошибок.



Рисунок 3.15 – Отчет о тестировании приложения

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была успешно разработана и реализована клиент-серверная архитектура современного многопользовательского чат-приложения. При проектировании и разработке системы особое внимание уделялось обеспечению высокой производительности, отзывчивости интерфейса и масштабируемости компонентов. Программный комплекс построен на базе разделения зон ответственности между бэкенд-сервером и фронтенд-клиентом, что позволило достичь гибкости управления данными и высокого уровня безопасности.

В качестве технологического стека для разработки серверной части был выбран фреймворк .NET 10 с использованием объектно-реляционного маппера Entity Framework Core и СУБД PostgreSQL. Архитектура сервера спроектирована на основе принципов Clean Architecture и шаблона CQRS с применением библиотеки MediatR, что обеспечило строгую изоляцию операций чтения и изменения данных. В качестве основного инструмента для обеспечения real-time взаимодействия и мгновенного обмена сообщениями была интегрирована технология SignalR, реализующая постоянное полнодуплексное сокет-соединение между клиентами и сервером. Для защиты пользовательских данных и авторизации запросов была внедрена аутентификация на базе JWT-токенов.

Клиентское приложение реализовано на базе современной библиотеки React с использованием компилируемой среды TypeScript, гарантирующей строгую типизацию и минимизацию ошибок в коде. Архитектура управления состоянием на фронтенде построена на базе легковесного стейт-менеджера Zustand, обеспечивающего синхронное обновление интерфейса и кэширование локальных данных. Дизайн приложения выполнен с использованием компонентного подхода.

В рамках практической реализации проекта был создан полнофункциональный программный продукт, поддерживающий динамическую авторизацию, создание групповых чатов, каналов и персональных диалогов. Разработана и отлажена комплексная система обработки сообщений, включающая их мгновенную отправку, синхронное обновление содержимого, а также функционал закрепления важных сообщений и ответов на них. Реализована оптимистичная отправка текстового контента на стороне клиента, что существенно повысило плавность и скорость отклика интерфейса. Работа завершена успешным тестированием всех модулей системы в условиях одновременной работы нескольких изолированных сессий пользователей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Доманов, А. Т. Стандарт предприятия / А. Т. Доманов, Н. И. Сорока. – Минск: БГУИР, 2024. – 178 с.
- [2] ASP.NET Core Documentation [Электронный ресурс] / Microsoft. – Режим доступа : <https://learn.microsoft.com/en-us/aspnet/core/>. – Дата обращения : 20.04.2026.
- [3] SignalR Documentation [Электронный ресурс] / Microsoft. – Режим доступа : <https://learn.microsoft.com/en-us/aspnet/core/signalr/>. – Дата обращения : 21.04.2026.
- [4] PostgreSQL Documentation [Электронный ресурс] / PostgreSQL Global Development Group. – Режим доступа : <https://www.postgresql.org/docs/>. – Дата обращения : 21.04.2026.
- [5] Entity Framework Core Documentation [Электронный ресурс] / Microsoft. – Режим доступа : <https://learn.microsoft.com/en-us/ef/core/>. – Дата обращения : 20.04.2026.
- [6] Martin, R. C. Clean Architecture : A Craftsman's Guide to Software Structure and Design / R. C. Martin. – Boston : Pearson, 2017. – 432 p.
- [7] CQRS Pattern [Электронный ресурс] / Microsoft Azure Architecture Center. – Режим доступа : <https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs>. – Дата обращения : 23.04.2026.
- [8] MediatR [Электронный ресурс]. – Режим доступа : <https://mediatr.io/>. – Дата обращения : 23.04.2026.
- [9] React Documentation [Электронный ресурс]. – Режим доступа : <https://react.dev/>. – Дата обращения : 02.05.2026.
- [10] ASP.NET Core Identity and Authentication [Электронный ресурс] / Microsoft. – Режим доступа : <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/>. – Дата обращения : 20.04.2026.
- [11] Fowler, M. Patterns of Enterprise Application Architecture / M. Fowler. – Boston : Addison-Wesley, 2002. – 560 p.
- [12] Unit testing C# with xUnit [Электронный ресурс] / Microsoft. – Режим доступа : <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-csharp-with-xunit>. – Дата обращения : 15.05.2026.